Data Structures

Lecture 7

Linked List

Abstract Data Type

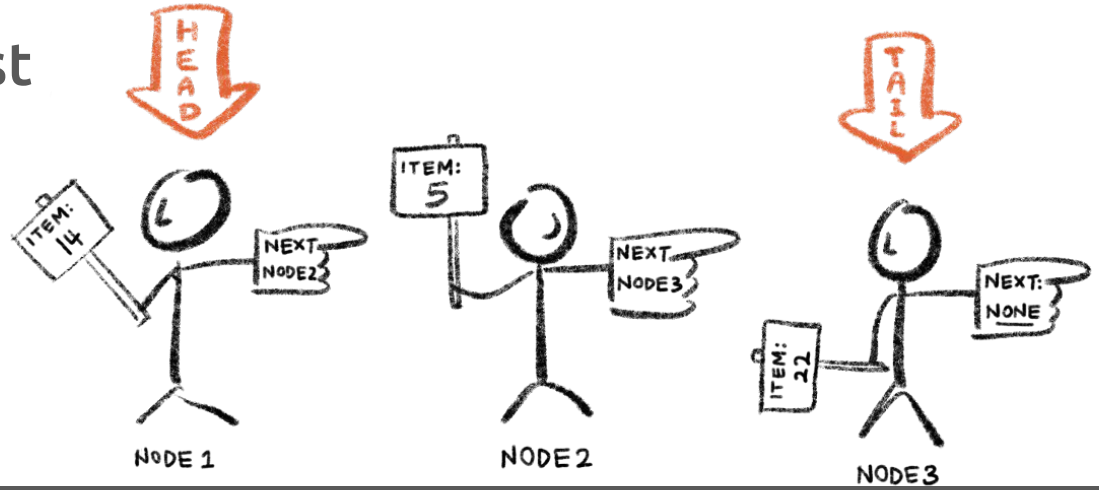
References:
Data Structures and Algorithms in Java
Michael T. Goodrich and Roberto Tamassia.

University of Kufa
Faculty of Computer Science
and Mathematics

Dr. Munqath Al-Attar
Information Technology Research and
Development Center ITRDC
Email:
munqith.alattar@uokufa.edu.iq
Website:
http://staff.uokufa.edu.iq/profile.html?munqith.alattar

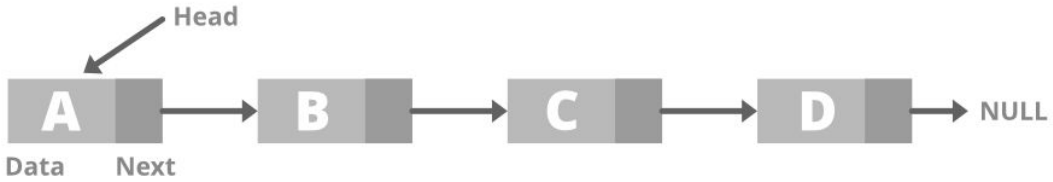
Topics

- Singly Linked Lists
  - Linked List Operations
- Circularly Linked List
- Doubly Linked List



Data Structures
Singly Linked Lists

- **Singly Linked list** is a collection of nodes that collectively form a **linear sequence**.
- Generally, each node stores an data element and a reference to the next node of the list.



- **Head**: a reference to the **first** node of the list, used to locate the node and any others.
- **Tail**: The **last** node of the list. It has null as its next reference.
    - The tail of a list can be found by **traversing** the linked list

Singly Linked List Operations

- **Traversal** - access each element of the linked list
- **Insertion** - adds a new element to the linked list
  - Inserting an Element at the **Head** of a Singly Linked List
  - Inserting an Element at the **Tail** of a Singly Linked List
  - Inserting an Element at the **Middle** of a Singly Linked List
- **Deletion** - removes the existing elements

Traverse a Linked List

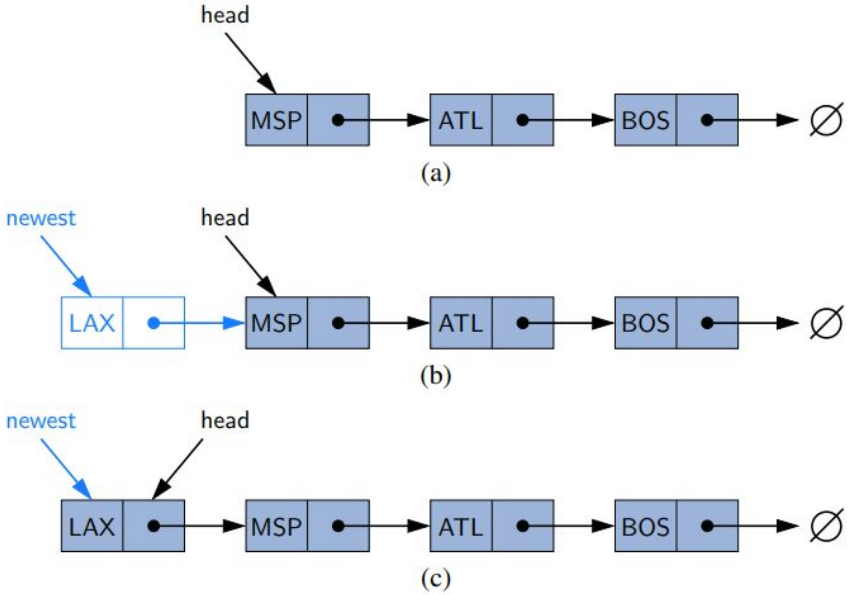
Used to visit each node of the list to perform an operation.

- We keep moving to the next nodes and display its contents.
- When we reach a node that its next reference is NULL, then we reached the end of the linked list.

```
Step 1: [INITIALIZE] SET PTR = HEAD
Step 2: Repeat Steps 3 and 4 while PTR != NULL
Step 3:     Apply process to PTR -> DATA
Step 4:     SET PTR = PTR->NEXT
       [END OF LOOP]
Step 5: EXIT
```

Inserting an Element at the Head of a Singly Linked List

- create a new node,
- set its element to the new element,
- set its next link to refer to the current head node,
- set the list's head reference to point to the new node.

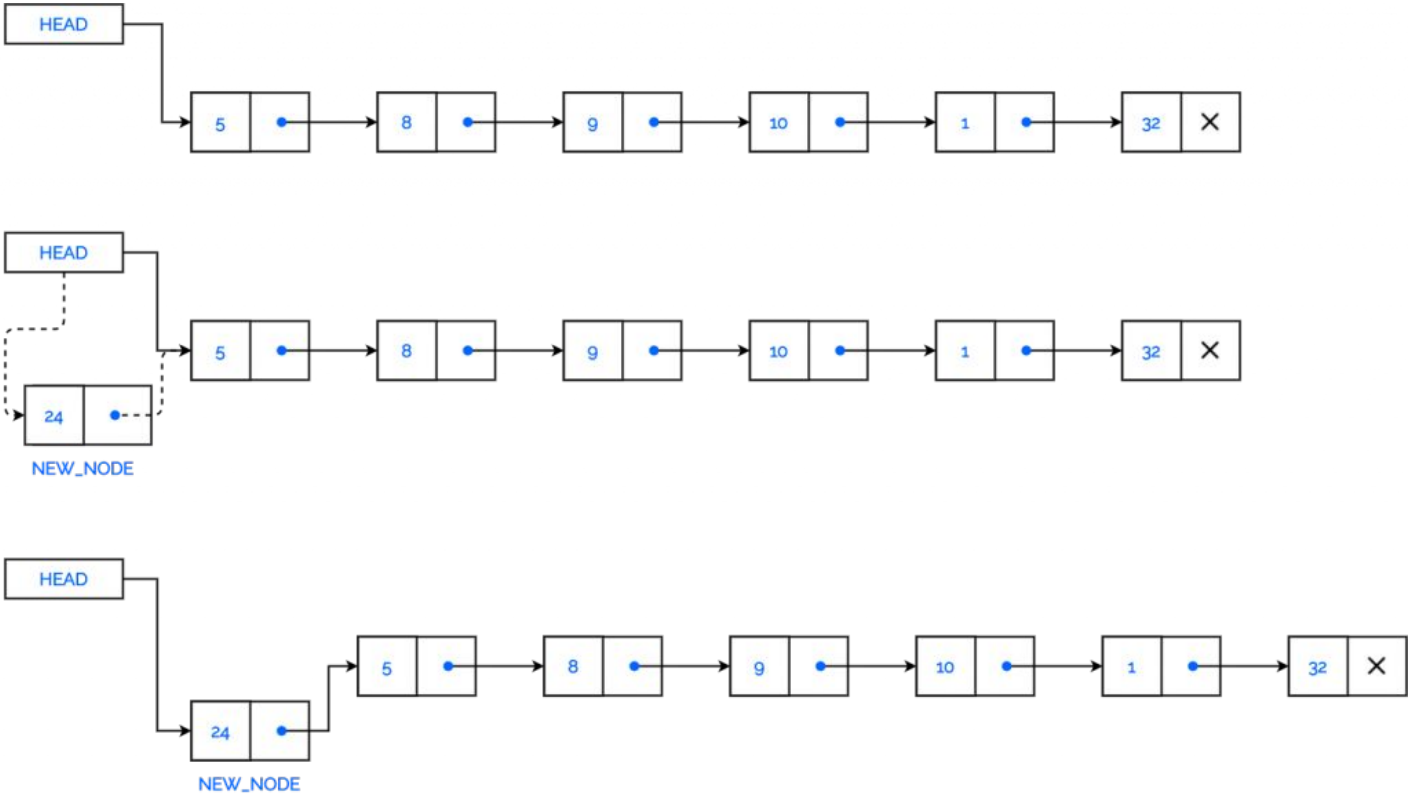


Algorithm addFirst(e):

```
newest = Node(e)   {create new node instance storing reference to element e}
newest.next = head   {set new node's next to reference the old head node}
head = newest   {set variable head to reference the new node}
size = size + 1   {increment the node count}
```

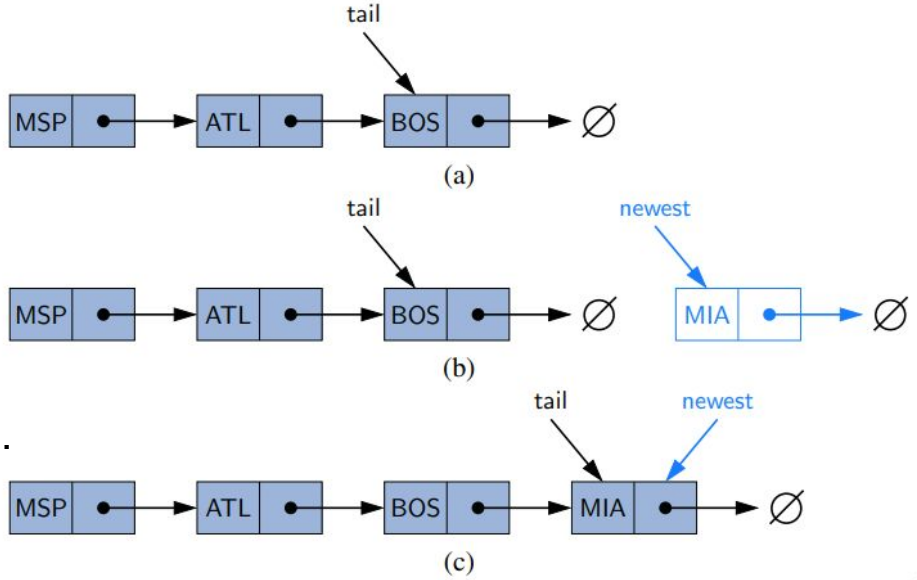
Data Structures

Linked List Operations: Inserting at the Head of a Singly Linked List



Inserting an Element at the Tail of a Singly Linked List

- create a new node,
- assign its next reference to null,
- set the next reference of the tail to point to this new node,
- update the tail reference itself to this new node.

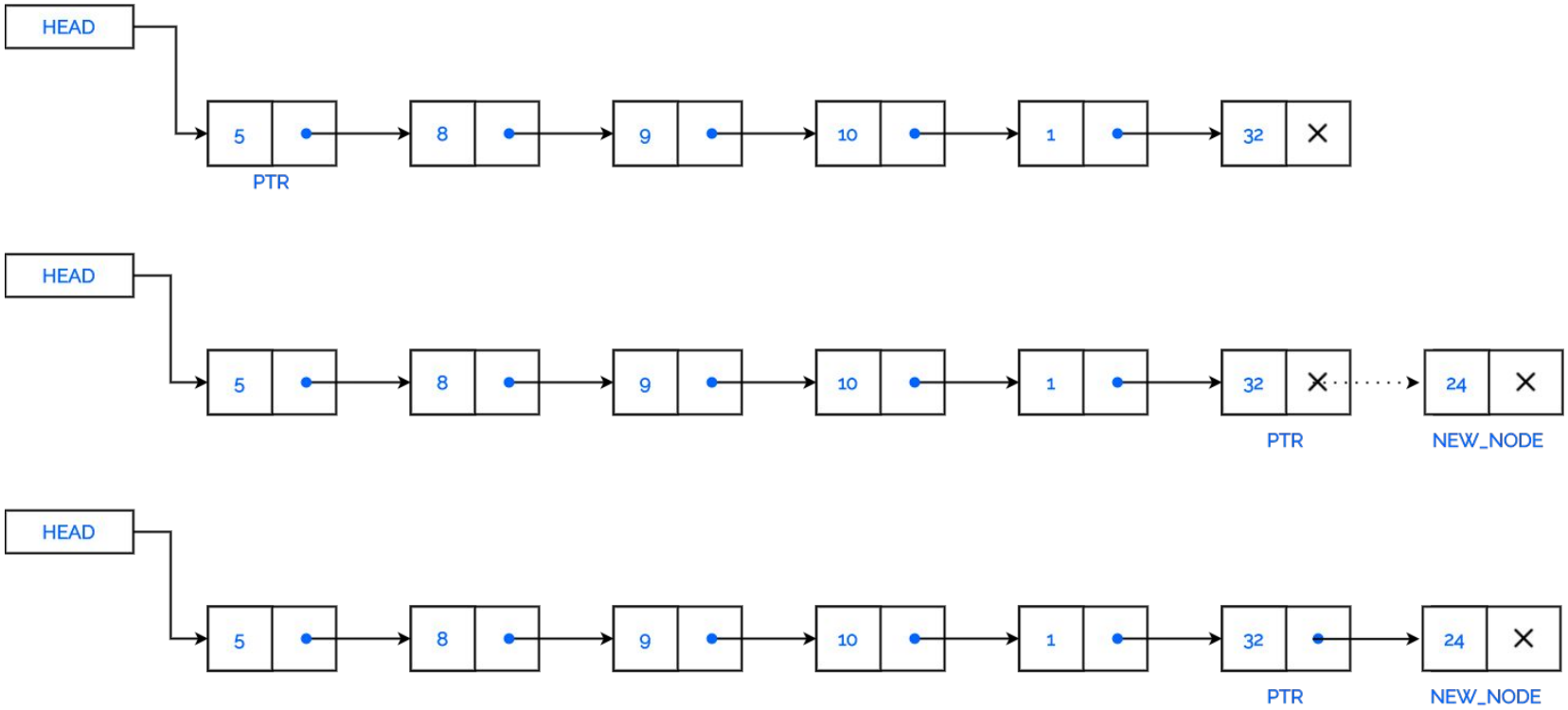


Algorithm addLast(e):

```
newest = Node(e)      {create new node instance storing reference to element e}
newest.next = null    {set new node's next to reference the null object}
tail.next = newest    {make old tail node point to new node}
tail = newest         {set variable tail to reference the new node}
size = size + 1       {increment the node count}
```

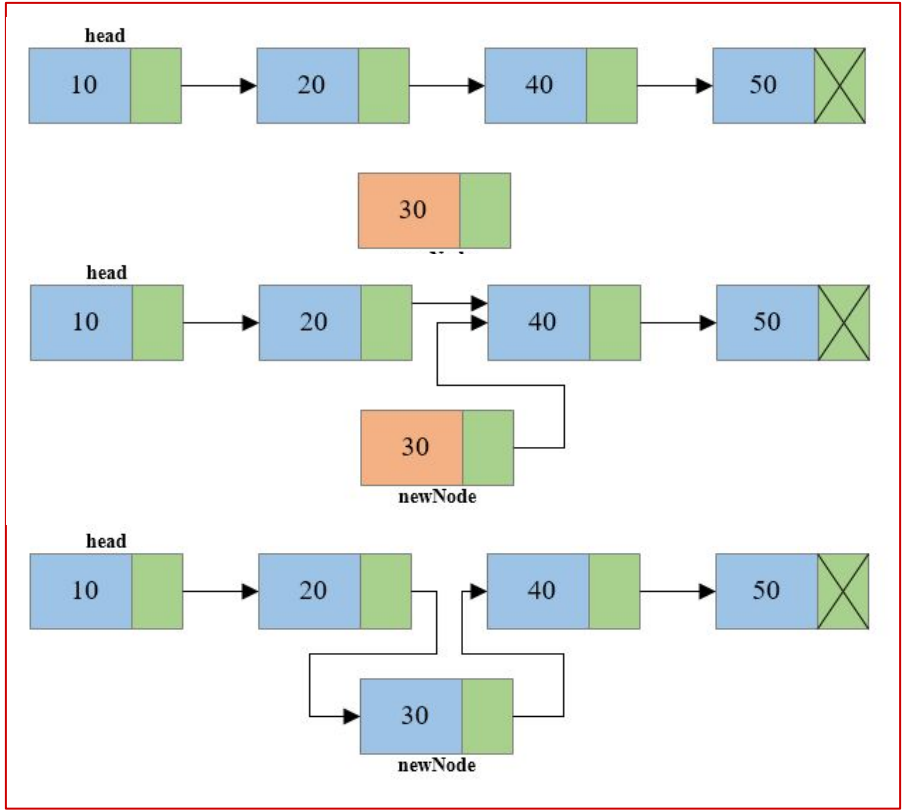
Data Structures

Linked List Operations: Inserting at the Tail of a Singly Linked List



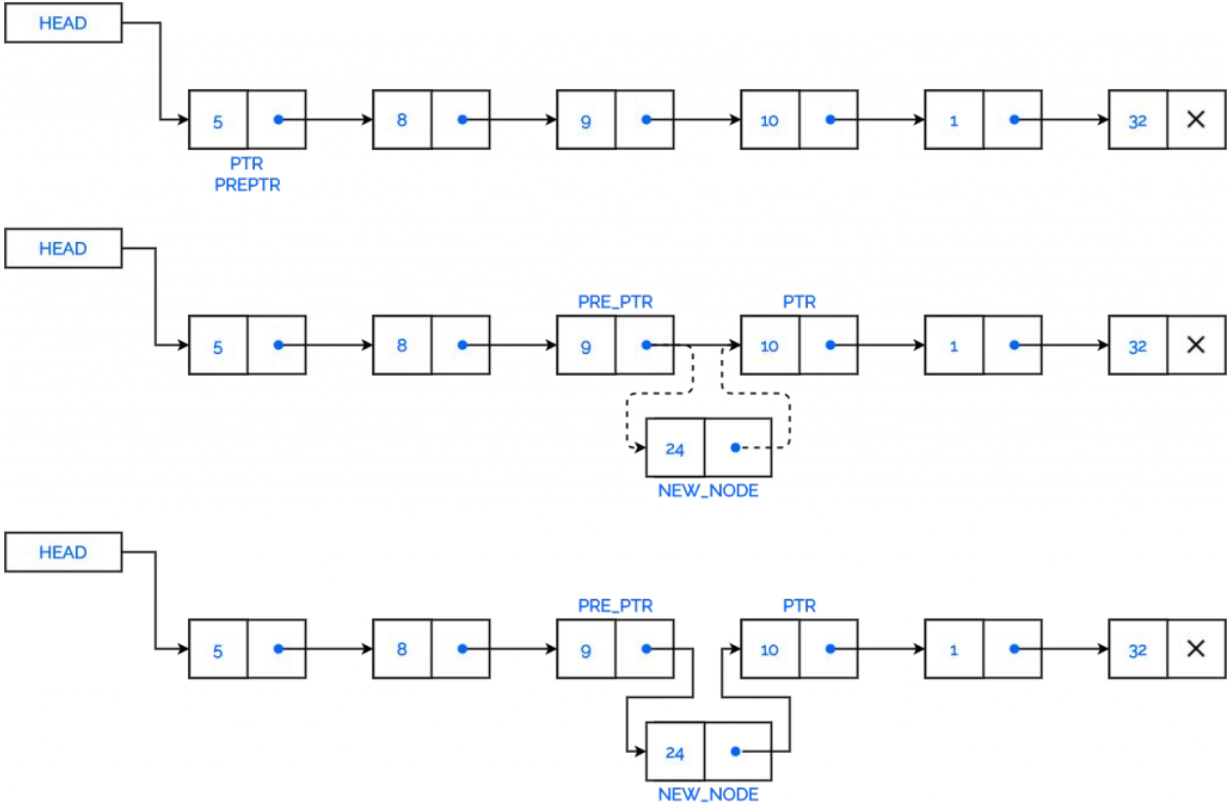
Inserting an Element at the Middle of a Singly Linked List

- Create a new node (to be inserted into the position *k*)
- Traverse to the *k-1*th node of the linked list
- set the next link of the new created node to refer to the *k*th node (like the *k-1*th node)
- set the next link of the *k-1*th node to the new created node



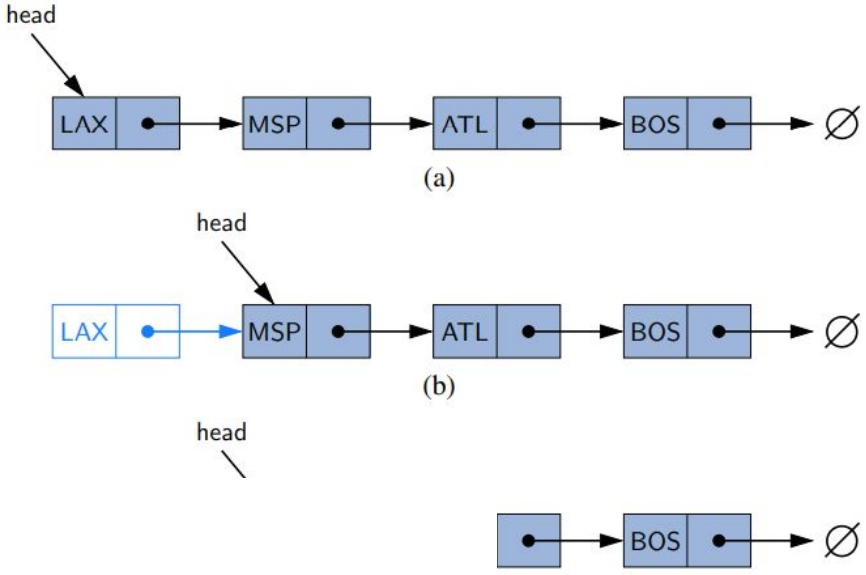
Data Structures

Linked List Operations: Inserting at the Middle of a Singly Linked List



Removing an Element from a Singly Linked List

- Removing an element from the head of a singly linked list is essentially the reverse operation of inserting a new element at the head.




```
Algorithm removeFirst( ):
    if head == null then
        the list is empty.
    head = head.next          {make head point to next node (or null)}
    size = size − 1           {decrement the node count}
```

- If first( ), last( ), or removeFirst( ) are called on a list that is empty, we will simply return a null reference and leave the list unchanged.

size( ):	Returns the number of elements in the list.
isEmpty( ):	Returns **true** if the list is empty, and **false** otherwise.
first( ):	Returns (but does not remove) the first element in the list.
last( ):	Returns (but does not remove) the last element in the list.
addFirst(e):	Adds a new element to the front of the list.
addLast(e):	Adds a new element to the end of the list.
removeFirst( ):	Removes and returns the first element of the list.

Unfortunately, it is not easy to delete the last node of a singly linked list:

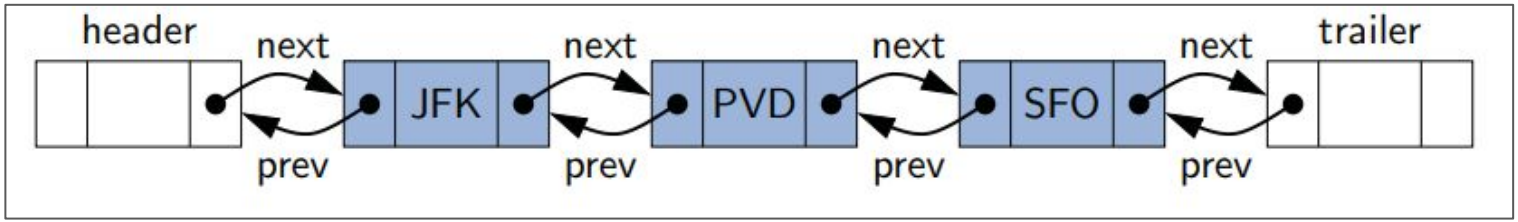
- Even if we set a tail reference directly to the last node, we must be able to access the node before the last node in order to remove the last node.
- But we cannot reach the node before the the last node by following next links from the tail.
- The only way to access this node is to start from the head of the list and go all the way through the list.
- But such a sequence of **link-hopping** operations could take a long time.

If we want to support such an operation efficiently, we use doubly linked list.

Doubly Linked List

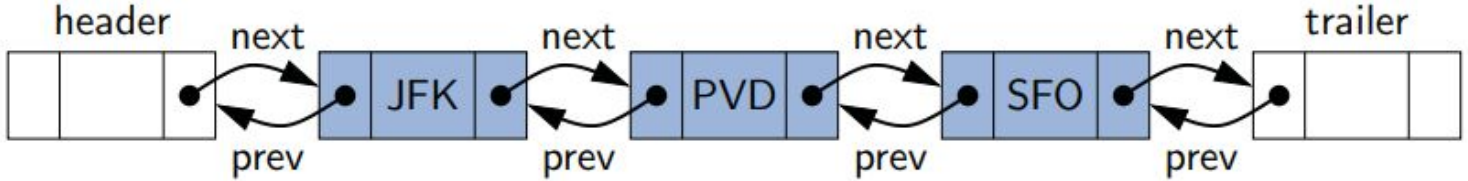
- To provide greater **symmetry**, we define a linked list in which each node keeps an explicit reference to the **node before** it and a reference to the **node after** it. Such a structure is known as a **doubly linked list**.

- “next” for the reference to the node that **follows** another,
- “prev” for the reference to the node that **precedes** it.



Data Structures
Doubly Linked List

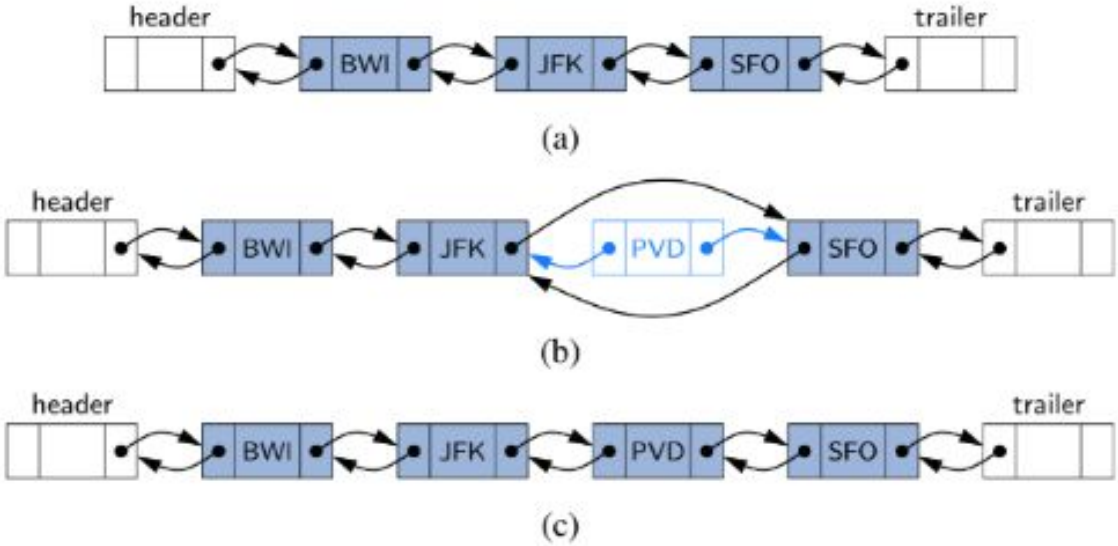
- **Header** node at the beginning of the list, **Trailer** node at the end of the list. These nodes are known as **sentinels** (or guards),
- When using sentinel nodes, **an empty** list is initialized so that the next field of the header points to the trailer, and the prev field of the trailer points to the header. The remaining fields of the sentinels are irrelevant (presumably null, in Java).
- For a nonempty list, the header's next will refer to a node containing the first real element of a sequence, and the trailer's prev references the node containing the last element of a sequence.



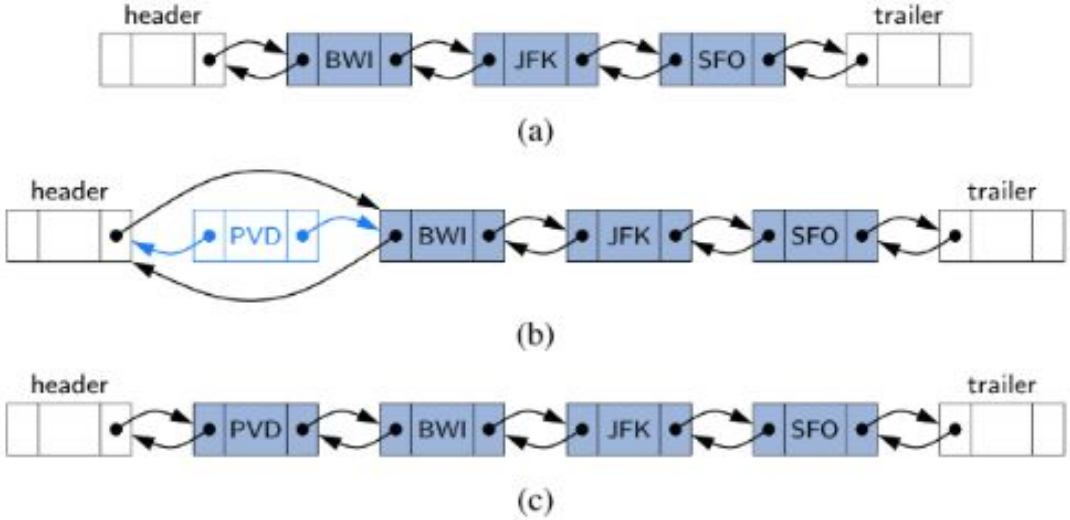
- The addLast in the SinglyLinkedList implementation required a condition to manage the special case of inserting into an empty list.
- In the general case, the new node was linked after the existing tail.
- But when adding to an empty list, there is no existing tail; instead it is necessary to reassign head to reference the new node.
- The use of a sentinel node in that implementation would eliminate the special case, as there would always be an existing node (possibly the header) before a new node.

Inserting with a Doubly Linked List

- Every insertion into doubly linked list will take place between a pair of existing nodes,

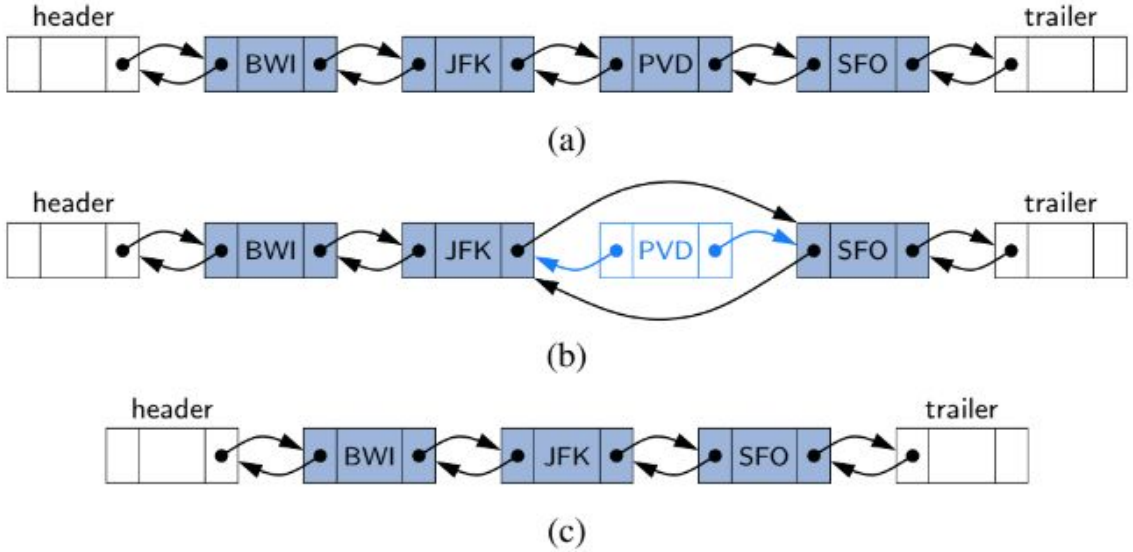


- For example, when a new element is inserted at the front of the sequence, we will simply add the new node between the header and the node that is currently after the header.



- To delete a node, the two neighbors of the node to be deleted are linked directly to each other. As a result, that node will no longer be considered part of the list.

- Because of sentinels, the same implementation can be used when deleting the first or the last element of a sequence, because even such an element will be stored at a node that lies between two others.



size( ):	Returns the number of elements in the list.
isEmpty( ):	Returns **true** if the list is empty, and **false** otherwise.
first( ):	Returns (but does not remove) the first element in the list.
last( ):	Returns (but does not remove) the last element in the list.
addFirst(e):	Adds a new element to the front of the list.
addLast(e):	Adds a new element to the end of the list.
removeFirst( ):	Removes and returns the first element of the list.
removeLast( ):	Removes and returns the last element of the list.

If first( ), last( ), removeFirst( ), or removeLast( ) are called on a list that is empty, we will return a null reference and leave the list unchanged.

There are many applications in which data can be more naturally viewed as having a cyclic order, with well-defined neighboring relationships, but no fixed beginning or end.

For example:

- Many multiplayer games are turn-based, with player A taking a turn, then player B, then player C, and so on, but eventually back to player A again, and player B again, with the pattern repeating.
- City buses and subways often run on a continuous loop, making stops in a scheduled order, but with no designated first or last stop per se.

- Most operating systems allow processes to effectively share use of the CPUs, using some form of an algorithm known as **round-robin scheduling**. A process is given a short turn to execute, known as a time slice, but it is interrupted when the slice ends, even if its job is not yet complete. Each active process is given its own time slice, taking turns in a cyclic order.

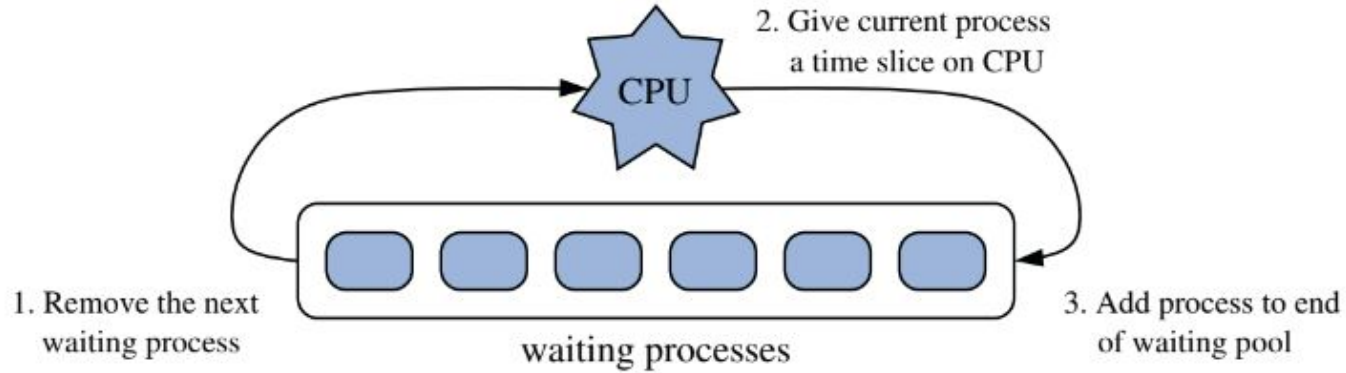


Figure 3.15: The three iterative steps for round-robin scheduling.

A circularly linked list structure is a singularly linked list in which the next reference of the tail node is set to refer back to the head of the list (rather than null),

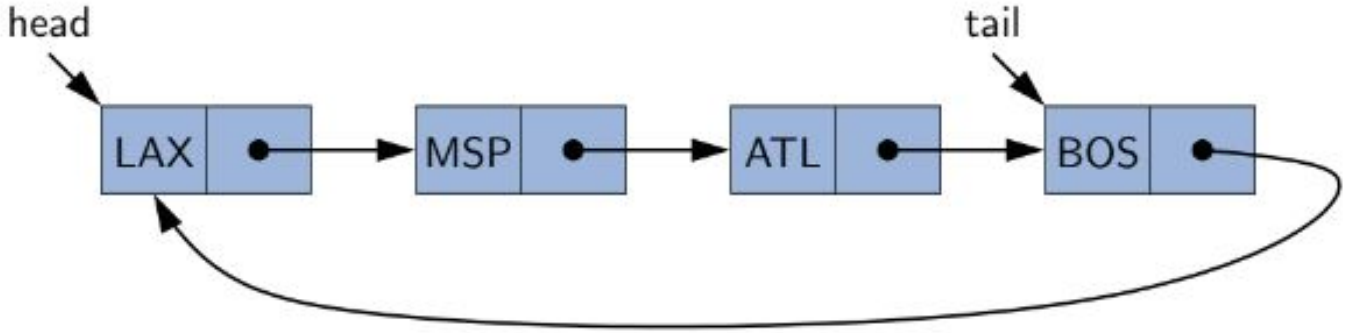


Figure 3.16: Example of a singly linked list with circular structure.

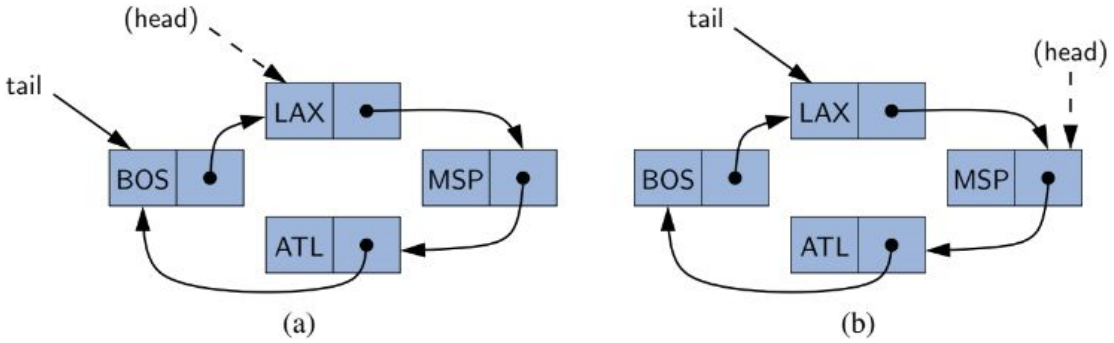
All of the behaviors of SinglyLinkedList class are supported in the implementation of CircularlyLinkedList class, and one additional update method:

rotate( ): Moves the first element to the end of the list.

Additional Optimization

- It is not required to explicitly define the head reference.
- As the reference to the tail is defined, we can locate the head as tail.getNext( ).
- Defining only the tail reference
  - saves memory usage,
  - Makes the code simpler and more efficient, (it removes the need to perform additional operations to keep a head reference.

To move around, we make the tail reference to point to the node that follows it (the head of the list).



Data Structures
Circularly Linked List: Operations

To add a new element at the front of the list:

- create a new node
- link it just after the tail of the list.

To implement the addLast method, we can rely on the use of a call to addFirst and then immediately advance the tail reference so that the newest node becomes the last.

Removing the first node by updating the next field of the tail node to bypass the implicit head.